

F2F Academy

FINAL PROGRAM REVIEW — LASER 2026

PROGRAM	LASER — Farm2Facts Student Developer Program, 2026 cohort
PLATFORM	Farm2Facts (F2F) — Vue 3 frontend, Rails 6.1 + MySQL backend
INTERNS	Bright, Logan
PROGRAM STAFF	Alfonso Morales, Garrett Smith, Abanish Khatry
MATERIALS	F2F Academy (Jekyll site) — onboarding plan, workflow guides, sprint task guides, study materials
STATUS	Program complete. Task 1 delivered in full; Task 2 delivered through sub-task 2.1, remainder paused.

1. Purpose of This Document

This is the closing record for the LASER 2026 program. It exists so that anyone reading it once — a lab director, a future program lead, or the next cohort's supervisor — comes away with a full picture of what was built, what the interns actually did, and where the program stopped and why.

The task cards that carried this content on the F2F Academy home page have been retired now that the program has wrapped. Everything they described is captured here.

2. Program Overview

LASER was an onboarding program for student developers joining Farm2Facts, a UW-Madison initiative that supports farmers markets by collecting vendor, attendance, and sales data and modeling food access across communities. The goal was not to hand the interns tickets. It was to take two students with no prior exposure to the codebase and move them, in scaffolded steps, to the point where they could contribute to a real two-repo application using LLM-assisted development practices.

Work was organized into named sprints of roughly two weeks each. Each sprint had one task guide published on the F2F Academy site, and each guide was self-contained: concepts, instructions, file templates, submission checklist, and a closing summary of skills gained.

Sprint	Weeks	Task Guide	Outcome
Welcome to the Barn	1 & 2	Task 1 Guide	Completed in full by both interns
Cheese Curds & Code	3, 4 & 5	Task 2.1 Guide (APIs for Beginners)	Sub-task 2.1 completed by both interns; the follow-on module was paused

Both sprints ran against the live Farm2Facts frontend repository on UW-Madison's GitLab, so the interns were working in the same repo, on the same branching model, and through the same review process as the professional developers on the project.

3. How the Tasks Were Built

Both task guides were written to the same template. That consistency was deliberate: once an intern learned how to read one guide, they could read any of them, and the mechanical parts of contributing (branch, directory, commits, merge request) stopped being a source of friction after the first sprint.

3a. The Guide Template

Every guide is a stack of collapsible cards, numbered in the order the intern works through them:

Card	What it does
Setup	Branch off <code>dev</code> with a prescribed branch name, create a prescribed directory structure, and commit each deliverable separately with a prescribed commit message. Nothing is left to interpretation.
Concepts / Learning	The material the intern needs before starting, written into the guide itself rather than only linked out, plus an ordered list of external resources.
Assignments	One card per deliverable. Each states duration, difficulty, step-by-step instructions, and the exact questions to answer.
Templates	A fill-in-the-blank Markdown skeleton for each written deliverable, so the intern spends their effort on the thinking, not the formatting.
PR Creation & Submission	A pre-submission checklist, the expected file tree, and click-by-click GitLab merge request steps. Work is not submitted until the MR is open and assigned.

Card	What it does
What You Learned	Skills gained, grouped and tagged, so the intern can see what carries forward into the next sprint.

3b. Design Decisions Behind the Template

- **One deliverable, one commit.** Both guides list a table of expected commits with exact messages. This taught commit hygiene by making it the only way to follow the instructions, rather than by lecturing about it.
- **Deliverables live in the repo.** Written work went into `task1_laser/laser_[firstName]/` and `task2_laser/laser_[firstName]/` inside the frontend repo, not into a document folder. Reading, writing, and reviewing all happened in the same place the code lives.
- **The merge request is the finish line.** Every guide closes on the MR. An intern who wrote everything but never opened the MR had not finished the task, by design.
- **Claude Code as a first-class tool, not a shortcut.** Task 1 asks interns to use Claude Code to explore the codebase, then explicitly requires them to rewrite the output in their own words.
- **Warm up on something you cannot break.** Task 2.1 starts on a practice API before touching the real one, so a mistake in the learning phase has no consequences.

3c. Supporting Materials

The task guides did not stand alone. Each leaned on shared material published alongside them:

- **Student Onboarding Plan** — Git and GitLab concepts, links to both repos, and database access, read before any task begins.
- **Frontend and Backend Guides** — clone, configure, and run-locally walkthroughs for the Vue 3 frontend and the Rails 6.1 + MySQL backend, with separate macOS and Windows paths.
- **Workflow Guides** — how we work, the Git workflow reference (branch strategy, commit rules), and the pull request guide (title format, description template, reviewer assignment).
- **Study Materials** — a sprint-mapped index of readings and documentation.
- **Curriculum & Task List** — the sprint table linking each sprint to its guide and to related coursework (CS200, CS300, CS571 for Sprint 1).

4. Task 1 — “Welcome to the Barn”

SPRINT	Welcome to the Barn
WEEKS	1 & 2

SOFT DEADLINE	Thursday, June 25
BRANCH	<code>feat/[firstName]_task1</code> , cut from <code>dev</code> in the frontend repo
STATUS	Completed in full by both interns.

4a. What the Task Was For

Task 1 answered a single question: can this person get the project running, find their way around it, and submit work the way the team submits work? It covered environment setup, codebase orientation, tooling, project context, and the full contribution workflow, and it produced four artifacts that made each of those visible.

4b. What It Covered

Card 1 — Onboarding. Four sub-sections walked the intern from zero to a working branch:

- *1.1 Get the repos running locally* — read the Student Onboarding Plan, then set up the Vue 3 frontend and the Rails 6.1 backend using the per-repo guides.
- *1.2 Open your feature branch* — branch from `dev` , then create the `task1_laser/`
`laser_[firstName]/` directory that holds all four deliverables.
- *1.3 Committing your work* — imperative commit messages under 72 characters, with a table giving the exact message for each of the four deliverables.
- *1.4 Codebase understanding report* — the first substantive deliverable, described below.

The codebase overview. The intern used Claude Code to explore ten directories under `src/views/` in the frontend repo and wrote a short *Contains* and *Role* description for each, from a supplied Markdown template:

Full description required	High-level overview only
• <code>DownloadDocuments</code> • <code>LandingPage</code> • <code>MarketOrg</code> • <code>MemberResources</code> • <code>SelectMetrics</code>	• <code>Analysis</code> • <code>Instrument</code> • <code>Instrument_MarketOrg</code> • <code>Profiles</code> • <code>Users</code>

Scoping the last five to an overview was intentional. The point was a usable map of the frontend, not an exhaustive audit, and capping the depth kept a two-week task from becoming a month-long one.

Card 2 — Claude Code 101. Interns completed Anthropic's Claude Code 101 course and submitted the completion certificate. The card then went beyond the course: it explained what a `CLAUDE.md` file is and why the frontend repo has one, had the intern read the repo's existing file, and then assigned them to write their own personal `Claude_[firstName].md` reflecting their working style and what they had learned about the codebase so far. The final step was comparative: run a session against their own file, run one against the repo's file, and note the difference in how Claude behaved.

Card 3 — The F2F article. Interns read *Citizen Scientist: Farm 2 Facts Supporting Farmers Markets* (Ledesma et al., 2021) and wrote a 2–3 paragraph response to three questions: what problem F2F solves for market managers and why it existed before F2F; one case study from the article (South Milwaukee Downtown Market, Hope and Main, Brown Deer, or ACEFM) and what decision the collected data supported; and, as a developer, what part of the F2F system they would most want to improve. The third question deliberately connects the research context back to engineering intent.

Card 4 — PR creation and submission. A pre-flight checklist confirmed the branch name, the directory, and all four files. The card then explained that GitLab calls it a Merge Request but the workflow is identical to a GitHub Pull Request, and gave the six-step path through the GitLab UI, targeting `dev` and filling out every field of the description template.

4c. Deliverables

File	Content
<code>codebase_overview.md</code>	Contains/Role write-up for all ten <code>src/views/</code> directories
<code>claude_code_101_certificate.pdf</code>	Anthropic course completion certificate
<code>f2f_article_response.md</code>	Response to the three article questions
<code>Claude_[firstName].md</code>	Personal Claude Code configuration file

All four in `task1_laser/laser_[firstName]/`, each with its own commit, submitted as one merge request into `dev`.

4d. Skills Established

Area	Covered
Git fundamentals	Staging and commits, <code>git status</code> / <code>git diff</code> , local vs. remote
Git branching	Feature branch isolation, branching from <code>dev</code> , the four-level branch structure

Area	Covered
Pull requests	The PR as the unit of review, imperative titles, the description template
Codebase navigation	<code>src/views/</code> layout, directory-first reading, using Claude as a navigator
CLAUDE.md	What it does, writing context prompts, comparing session outputs

4e. Result

Task 1 was completed in full. Both interns set up both repos locally, produced all four deliverables, committed them individually, and submitted merge requests into `dev`. This is the one sprint of the program that ran end to end exactly as designed.

5. Task 2 — “Cheese Curds & Code”

SPRINT	Cheese Curds & Code
WEEKS	3, 4 & 5
STRUCTURE	Module 1 — APIs for Beginners (Task 2.1). Module 2 — automated data extraction and platform integration.
STATUS	Task 2.1 completed by both interns. Module 2 paused.

5a. How the Sprint Was Structured

Task 2 was the bridge from onboarding to real feature work. It was split into two modules. Module 1 (Task 2.1) taught APIs from the ground up with no code required, using the Wisconet API — Wisconsin's statewide environmental sensor network — as the live data source. Module 2 was to build on that foundation with code that automates data extraction and feeds it into the Farm2Facts platform.

Splitting the sprint this way meant the interns learned request/response mechanics against a read-only public API, where the worst case is a 404, before writing anything that touches the F2F codebase.

5b. Task 2.1 — What It Covered

Concepts, written into the guide. Before any assignment, the guide covered what an API is (the restaurant analogy: customer, waiter, kitchen), the REST style, and JSON. It broke a real Wisconet URL into its parts — protocol, base URL, endpoint path, action, query parameters — and gave a status code reference (200, 404, 422, 500). Only after that did it list external resources, in a required order: MuleSoft's *What is an API?*, the freeCodeCamp *APIs for Beginners* course, the first three modules of

Postman API Fundamentals, the REST API Tutorial, W3Schools on JSON, and finally the Wisconet API documentation.

Assignment 1 — API Detective (~30 min, beginner). Browser only, no tools. Interns opened the Wisconet stations endpoint, found the station nearest their home county, built a URL for it by hand, and fetched its latest measures. Seven questions followed: station ID and county, latitude and longitude, how many fields the endpoint returned, which field name holds air temperature, whether the station has soil sensors and at how many depths, what `collection_frequency` means, and — in their own words — the difference between the station list endpoint and the latest-measures endpoint.

Assignment 2 — Postman Explorer (~45 min, beginner). Interns installed Postman Desktop and warmed up on JSONPlaceholder, a public practice API: fetch all users, fetch one user by ID, then filter posts by `userId` using the Params tab. They then moved to Wisconet: all stations, the ALTN (Arlington) station detail and its elevation, and ALTN's latest measures with the response time noted. All three were saved into a Postman Collection named “Wisconet Exploration.” The deliverable asked for a screenshot of the collection, the status code of each request, what happens with a station ID that does not exist, and a short explanation of path parameters versus query parameters.

Assignment 3 — Soil Temperature Deep Dive (~60 min, intermediate). The first assignment with real analytical weight. Interns learned the Wisconet field naming convention — `{frequency}_{measure}_{units}_{aggregation}@{depth}` — converted calendar dates into Unix timestamps, and built a Postman request against the measures endpoint for station HNCK pulling soil temperature at five depths (2in, 4in, 8in, 20in, 40in) across a 24-hour window. They repeated it for a second station and then switched from 5-minute readings to daily maximums to see what changed. Five questions closed it out: the readings at 2in, 8in, and 40in and what pattern appears as depth increases; which of their two stations has warmer soil at 4in and a hypothesis as to why; how many 5-minute readings a 24-hour window should produce and whether their count matches; what `preceding_value` represents; and what they would change in the field name to get hourly data.

Assignment 4 — Document Your Understanding (~30 min, reflection). Three parts. Part A, a one-page plain-English summary of the Wisconet API: what data it provides, how it is structured, what a soil temperature request looks like described in words, and what limitations they noticed. Part B, a hand-drawn or slide-drawn request-response flow diagram with the browser or Postman, the API server, the database, and the JSON response, every arrow labeled. Part C, a proposed use case: a tool for Wisconsin farmers built on this API, naming the user, the problem, and the endpoints it would use. Part C was written as the on-ramp into Module 2.

5c. Task 2.1 Deliverables

File	Content
<code>assignment1_api_detective.md</code>	Seven answers from browser-only exploration

File	Content
<code>assignment2_postman_explorer.md</code>	Collection screenshot, status codes, error behavior, parameter types
<code>assignment3_soil_temperature.md</code>	Multi-depth, multi-station soil temperature findings
<code>assignment4_summary.md</code>	API summary, request-response diagram, proposed use case

All four in `task2_laser/laser_[firstName]/` on branch `feat/[firstName]_task2`, each with its own commit, submitted as one merge request into `dev`. A companion reference packet, *LASER Intern API Learning Packet — Module 1 of 2*, carried the same material in long form with a field reference and cheatsheet appendix.

5d. Skills Established

Area	Covered
API fundamentals	REST architecture, HTTP methods, status codes
JSON	Objects and arrays, key-value pairs, reading responses
URL structure	Base URL, path parameters, query parameters
Tooling	The browser as an API client, Postman requests, Postman Collections
Wisconet API	Station endpoints, field naming convention, Unix timestamps

5e. Result and Pause

Task 2.1 was completed by both interns. They worked through the concepts and all four assignments, pulled live soil data from the Wisconet network at multiple depths and stations, and submitted their deliverables through the same branch-commit-merge-request workflow established in Task 1.

Module 2 did not run. The sprint was paused after 2.1 because of an unresolved licensing issue affecting the software the integration work depended on. This was a blocker outside the interns' control and unrelated to their progress: the learning module had already landed, and the interns were ready for the code module. Rather than start work that could not be finished or merged, the sprint was stopped at a clean boundary — a completed, reviewable sub-task — and the remaining scope was set aside.

6. What Module 2 Was Scoped to Be

The follow-on work was fully specified before the pause, in the intern project memo *MesoNet Soil Metrics & Notification System*. Recording it here means the scope is not lost if the licensing situation is resolved and the work is picked back up.

6a. Feature Scope

- **Data integration.** Consume the MesoNet API for soil temperature, soil moisture, and rainfall near a user's farming area; surface monthly high/low summaries on both Farmer and Market/Vendor profile dashboards.
- **Threshold-based alerts.** Let users set custom upper and lower bounds for soil temperature and moisture, with email notification when a threshold is crossed in either direction, plus a profile-level opt-in and customization flow.
- **Weekly digest.** A scheduled cron job emailing opted-in users a soil temperature summary relevant to their region.
- **Reference data.** Use readings from the Arlington Agricultural Research Station (UW-Extension, Arlington, Wisconsin) to derive sensible default threshold values.

6b. Planned Assignments

Intern	Focus	Responsibilities
Bright	SWE-focused	• MesoNet API integration (requests, auth, parsing) • Documentation of endpoints and data models • Profile integration for both user types • Threshold alert logic and email triggers • Cron job for the weekly digest
Logan	SWE + research	• All SWE tasks, collaboratively with Bright • Deep dive into Arlington experimental farm data via UW-Extension • Research agricultural metrics and derive threshold defaults • Propose new F2F platform features from those findings

The planned timeline was three weeks: API setup and integration, then profile display and alerts, then cron jobs, research, and polish.

7. Completion Summary

Unit	Status	Notes
Task 1 — Welcome to the Barn	Complete	All four deliverables submitted by both interns via merge request into <code>dev</code> . Ran exactly as designed.
Task 2.1 — APIs for Beginners	Complete	All four assignments submitted by both interns. Live Wisconet data pulled and analyzed.
Task 2, Module 2 — MesoNet integration	Paused	Not started. Blocked by an unresolved licensing issue affecting the software the work depended on. Scope documented and preserved.

Every task that was published to the interns was completed by the interns. Nothing was left unfinished on their side.

8. Takeaways and What Carries Forward

8a. What Worked

- **Prescriptive setup cards removed friction.** Fixing the branch name, directory structure, and commit messages in advance meant the interns spent Task 2 thinking about APIs rather than re-learning Git.
- **Deliverables as written artifacts made progress legible.** Every task produced reviewable files in the repo, so completion was never ambiguous.
- **Warm-up-then-real sequencing.** JSONPlaceholder before Wisconet, and browser before Postman, kept the difficulty curve gentle without lowering the ceiling.
- **Pausing at a clean boundary.** Because the sprint was split into modules, the licensing block cost the program a module rather than a half-finished feature branch.

8b. What to Address Next Time

- **Confirm licensing and access before scoping.** The single thing that stopped this program was an external dependency that was not cleared in advance. Verifying that every tool and service in a sprint's critical path is licensed and accessible should be a precondition for publishing the guide.
- **Keep a no-dependency fallback module.** A parallel track that needs nothing but the public repos would have kept the interns moving through the block.

8c. Reusable Assets

The following came out of LASER and are being carried into the WISCURDS program:

- The task guide template — setup, concepts, assignments, templates, submission, what you learned.
- The branch, directory, and commit conventions, now standard across programs.
- The API learning packet, reusable as standalone material for any cohort.
- The MesoNet project memo, holding the full unbuilt scope.
- The F2F Academy site structure itself: per-program blocks, collapsible task cards, and a shared core materials section.

Farm2Facts @KaufmanLab • F2F Academy • LASER 2026 Final Program Review